

Rapport de Projet de Fin d'Année

Conception et réalisation d'une plateforme web de gestion des ressources humaines

Projet Django basé sur le dépôt HR_django

Plateforme RH intégrée

Employés, congés, demandes administratives, pointage, planning, tâches, support RH, formations, boutique, paie, notifications, audit, Brevo et Gemini.

Encadrant : Dr. Houda Orchi

Réalisé par : Zine Mohamed Amine
Ghaout Khalid

Année universitaire : 2025–2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à Dr. Houda Orchi pour son encadrement, sa disponibilité et ses conseils précieux tout au long de la réalisation de ce Projet de Fin d'Année. Son accompagnement nous a permis de mieux structurer notre travail, d'améliorer nos choix techniques et de renforcer la qualité globale de notre solution.

Nous la remercions également pour ses orientations méthodologiques, ses remarques constructives et son suivi rigoureux, qui ont contribué à l'avancement du projet dans de bonnes conditions. Ses recommandations nous ont aidés à mieux organiser notre démarche, depuis l'analyse des besoins jusqu'à la conception, la réalisation et la validation de l'application.

Nous adressons aussi nos remerciements à l'ensemble des personnes qui nous ont apporté leur aide, directement ou indirectement, durant la réalisation de ce travail.

Table des matières

Remerciements	i
Liste des figures	v
Liste des tableaux	vi
Introduction générale	1
1 Présentation du cadre de projet	3
1.1 Introduction	3
1.2 Présentation du contexte général	3
1.3 Étude de l'existant	4
1.3.1 Description de l'existant	4
1.3.2 Critique de l'existant	4
1.3.3 Solution proposée	5
1.4 Objectifs du projet	5
1.5 Choix du modèle de développement	6
1.6 Planning prévisionnel	6
1.7 Conclusion	7
2 Spécification des besoins	8
2.1 Introduction	8
2.2 Besoins fonctionnels	8
2.3 Besoins non fonctionnels	9
2.4 Présentation des acteurs	10
2.5 Description des principaux cas d'utilisation	10

2.6	Diagramme global des cas d'utilisation	11
2.7	Conclusion	11
3	Conception du système	13
3.1	Introduction	13
3.2	Vue fonctionnelle	13
3.2.1	Organisation des modules	13
3.2.2	Éléments retirés de la conception	14
3.3	Vue statique	14
3.3.1	Diagramme de classes métier principal	14
3.3.2	Classes assistant, tickets et comptes	15
3.4	Vue applicative	16
3.4.1	Architecture logicielle	16
3.5	Vue dynamique	17
3.5.1	Séquence création de compte	17
3.5.2	Séquence Planning vers Pointage	17
3.5.3	Séquence assistant Gemini	18
3.6	Choix technologiques	18
3.7	Conclusion	19
4	Réalisation du système	20
4.1	Introduction	20
4.2	Environnement de développement	20
4.2.1	Environnement matériel	20
4.2.2	Environnement logiciel	20
4.3	Architecture du projet	21
4.4	Implémentation des modules principaux	21
4.4.1	Authentification, comptes et Brevo	21
4.4.2	Gestion des employés et de l'organisation	21
4.4.3	Congés et demandes administratives	21
4.4.4	Planning avancé	22
4.4.5	Pointage intégré au Planning	22

4.4.6	Task Keep	22
4.4.7	Support RH par tickets	22
4.4.8	Administration	22
4.4.9	Assistant Gemini	23
4.4.10	Dashboard, navigation et UI/UX	23
4.5	Tests et validation	23
4.6	Principales interfaces graphiques	23
4.6.1	Tableau de bord	23
4.6.2	Organisation RH	24
4.6.3	Présence / Pointage	24
4.6.4	Planning	24
4.6.5	Tâches et Support	24
4.7	Déploiement	24
4.8	Difficultés rencontrées et solutions apportées	25
4.9	Conclusion	25
Conclusion générale		26
A Annexes		27
A.1	Routes principales	27
A.2	Éléments supprimés ou remplacés	28
A.3	Commandes utiles	28

Table des figures

2.1	Diagramme global des cas d'utilisation actualisé	11
3.1	Vue fonctionnelle par modules mères et onglets fils	14
3.2	Classes et services ajoutés ou renforcés	16
3.3	Architecture applicative actualisée	16
3.4	Séquence création de compte avec Brevo	17
3.5	Séquence du lien Planning et Pointage	17
3.6	Séquence de l'assistant Gemini sécurisé	18

Liste des tableaux

1.1	Planning prévisionnel actualisé du projet	6
2.1	Acteurs du système actualisé	10
2.2	Cas d'utilisation actualisés	10
3.1	Dictionnaire de données synthétique actualisé	15
3.2	Technologies utilisées dans le projet	18
4.1	Tests et validations réalisés	23
A.1	Extrait des routes applicatives actualisées	27
A.2	Éléments retirés de l'ancien rapport	28

Introduction générale

La gestion des ressources humaines constitue un axe central dans le fonctionnement d'une organisation moderne. Elle ne se limite plus à la conservation des dossiers du personnel ; elle couvre également le suivi des congés, le traitement des demandes administratives, la planification du travail, la mesure de la présence, la communication interne, la formation, les tâches d'équipe, le support RH, la boutique interne, les points, les notifications, l'audit et l'analyse salariale. Dans un environnement où les informations RH sont nombreuses et sensibles, l'utilisation de procédures manuelles ou dispersées entraîne des retards, des erreurs de suivi et une faible traçabilité.

Le projet étudié dans ce rapport est une plateforme web développée avec Django. Le dépôt analysé contient une application organisée autour de trois applications principales : `accounts`, `core` et `hr`. La solution couvre l'authentification, les rôles utilisateurs, le tableau de bord, les employés, les départements, les services, les postes, les congés, les demandes administratives, le pointage, le planning, les tâches, les formations, la messagerie RH transformée en support par tickets, les actualités, la boutique interne, les points, la paie, les notifications, l'audit, l'administration et l'assistant intelligent.

Cette version du rapport prend comme base le rapport PDF initial et le met à jour sans changer inutilement son orientation. Les modules encore présents sont conservés. Les descriptions devenues obsolètes sont retirées ou remplacées uniquement lorsqu'elles ne correspondent plus à l'application actuelle : le module Documents autonome est retiré car les pièces jointes sont désormais échangées dans les tickets, conversations et messages ; le module Réclamation RH autonome est remplacé par le support RH sous forme de tickets ; l'ancienne vérification prévue avec Microsoft Outlook est remplacée par Brevo ; l'ancienne intégration IA est remplacée par Gemini avec un fonctionnement RAG sécurisé.

L'objectif principal est donc de présenter une plateforme RH plus mature, plus professionnelle et plus proche d'un usage entreprise. L'application adopte une interface compacte, un système de navigation par onglets mères et onglets fils, des cartes KPI homogènes, des formulaires plus robustes, des validations backend, une meilleure gestion des exceptions, une séparation des responsabilités et des workflows métier plus complets.

Ce rapport est organisé en quatre chapitres. Le premier chapitre présente le cadre général du projet, l'existant, la solution proposée, les objectifs et le planning de réali-

sation. Le deuxième chapitre décrit les besoins fonctionnels, les besoins non fonctionnels, les acteurs et les principaux cas d'utilisation. Le troisième chapitre présente la conception, l'architecture, les modèles de données, les diagrammes et les choix technologiques. Le quatrième chapitre expose la réalisation, les modules développés, les tests, la validation, les interfaces, le déploiement et les difficultés rencontrées. Le rapport se termine par une conclusion générale et des annexes actualisées.

Chapitre 1

Présentation du cadre de projet

1.1 Introduction

Dans ce chapitre, nous présentons le contexte général du projet, l'étude de l'existant, les objectifs de la solution, le modèle de développement retenu et le planning prévisionnel. Cette partie reprend le cadre du rapport initial, mais elle l'actualise avec les modules réellement conservés et les fonctionnalités ajoutées lors de la dernière phase de développement.

1.2 Présentation du contexte général

Le projet correspond à une plateforme web de gestion des ressources humaines. L'application vise à regrouper dans un même environnement les traitements quotidiens liés aux employés et aux processus RH : gestion des dossiers, suivi hiérarchique, congés, demandes administratives, pointage, planning, tâches d'équipe, formations, messages RH, actualités, boutique interne, support par tickets, points et analyses salariales.

Le code source est organisé sous forme de projet Django. Le dossier `config` contient la configuration générale et le routage principal. L'application `accounts` assure l'authentification, les profils, les rôles, les demandes de création de compte, la vérification par courriel et la réinitialisation du mot de passe. L'application `core` porte le tableau de bord, l'administration et l'assistant global. L'application `hr` concentre les modèles métier, les formulaires, les vues, les services, les permissions, les templates, les tests et les commandes de données de démonstration.

L'évolution récente a renforcé la plateforme sur trois axes. Le premier axe concerne l'expérience utilisateur : interfaces plus compactes, filtres plus lisibles, sidebar plus stable, onglets mères et onglets fils, statistiques rapides et formulaires plus propres. Le deuxième axe concerne les workflows métier : Planning avancé, Pointage lié aux

shifts, tâches d'équipe, support RH par tickets, approbation des comptes, approbation des congés par manager et RH. Le troisième axe concerne la sécurité : vérification Brevo, reset du mot de passe par code, isolation des données selon les rôles, audit, refus des actions dangereuses par l'assistant et validations backend.

1.3 Étude de l'existant

1.3.1 Description de l'existant

L'existant manuel ou semi-numérique d'un service RH repose souvent sur des fichiers séparés, des échanges par courriel, des validations non centralisées et des historiques difficiles à exploiter. Une demande de congé peut être acceptée sans contrôle automatique du solde, une demande administrative peut rester sans réponse, un manager peut manquer de visibilité sur son équipe, ou une absence peut être calculée sans tenir compte du planning réel de l'employé.

Le rapport initial décrivait déjà une plateforme large. Toutefois, certaines parties ne correspondent plus à l'état actuel : les documents ne constituent plus un module isolé, les réclamations sont absorbées par les tickets RH, l'ancienne vérification Outlook n'est pas utilisée et l'assistant repose désormais sur Gemini. La mise à jour du rapport consiste donc à conserver la logique de base tout en corrigeant ces différences.

1.3.2 Critique de l'existant

L'existant présente plusieurs insuffisances : dispersion des données, absence de workflow unifié, faible traçabilité, difficulté de contrôle des droits, duplication des traitements, lenteur de validation, manque d'indicateurs et absence de lien direct entre planning et pointage. Dans le contexte d'un service RH, ces limites peuvent entraîner une perte de temps importante et une dégradation de la qualité de service envers les employés.

Les anciens choix techniques présentaient aussi des limites. Une intégration de vérification par Outlook aurait ajouté une dépendance inutile pour des courriels transactionnels. Une IA non filtrée par rôle aurait risqué d'exposer des informations sensibles. Un module Réclamation isolé aurait créé un doublon avec les conversations RH. Un module Documents autonome aurait multiplié les zones de dépôt, alors que les pièces jointes sont plus pertinentes dans leur contexte métier.

1.3.3 Solution proposée

La solution proposée est une application web Django centralisée, modulaire et sécurisée. Elle permet de gérer les processus RH depuis une interface unique, tout en séparant les responsabilités : authentification et comptes dans `accounts`, tableau de bord et administration dans `core`, traitements RH dans `hr`.

Le système proposé conserve les modules utiles du rapport initial et ajoute les évolutions suivantes :

- Planning complet avec vues calendrier, quotidienne, hebdomadaire, bihebdomadaire et mensuelle ;
- plans permanents, récurrences, pauses, conflits, rapports et API JSON ;
- Pointage lié au shift planifié pour calculer retards, sorties anticipées, heures manquantes et heures supplémentaires ;
- tâches d'équipe avec délégation manager, acceptation, validation, archives, feedback et points ;
- support RH sous forme de tickets avec pièces jointes, participants, statut, affectation RH, clôture, notation et récompenses ;
- création de compte avec vérification Brevo, approbation administrateur et onboarding RH ;
- mot de passe oublié avec code de vérification sécurisé ;
- assistant Gemini avec RAG, filtrage par rôle, refus des mutations et protection contre les injections de prompt ;
- administration par onglets enfants : utilisateurs, rôles, permissions, audit, sécurité, demandes de compte, rapports et paramètres.

1.4 Objectifs du projet

Les objectifs actualisés du projet sont les suivants :

- centraliser les opérations RH dans une application unique ;
- améliorer l'ergonomie par une interface compacte et professionnelle ;
- gérer correctement les employés, départements, services, postes et niveaux ;
- sécuriser les workflows de comptes par Brevo et approbation administrateur ;
- assurer la cohérence entre Planning et Pointage ;
- remplacer les réclamations isolées par un support RH professionnel ;

- remplacer le dépôt documentaire isolé par des pièces jointes dans les tickets et conversations ;
- offrir un assistant intelligent utile sans exposer les données sensibles ;
- renforcer les tests, validations, exceptions et contrôles d'accès.

1.5 Choix du modèle de développement

Le projet adopte une démarche incrémentale. Les migrations et les tests montrent une évolution progressive : modèle RH initial, ajout des notifications et de l'audit, enrichissement par les points, formations, boutique et rémunération, puis ajout du planning, du pointage, des tâches, du support par tickets, de la création de compte sécurisée et de l'assistant Gemini.

Cette progression correspond à une approche itérative adaptée à un projet métier. Chaque module peut être conçu, implémenté, testé et amélioré indépendamment. Les règles métier communes sont centralisées dans les services, les validations dans les formulaires ou modèles, les accès dans les permissions et les workflows utilisateur dans les vues.

1.6 Planning prévisionnel

TABLE 1.1 – Planning prévisionnel actualisé du projet

Phase	Travaux réalisés ou prévus
Étude préalable	Analyse du domaine RH, lecture du dépôt et identification des modules à conserver, ajouter ou retirer.
Analyse des besoins	Définition des acteurs, besoins fonctionnels, besoins non fonctionnels et cas d'utilisation.
Conception	Mise à jour des diagrammes pour intégrer Planning, Pointage, tâches, tickets, Brevo, administration et Gemini.
Réalisation	Développement Django des applications <code>accounts</code> , <code>core</code> et <code>hr</code> .
UI/UX	Passage vers des interfaces compactes, onglets mères et fils, KPI homogènes, filtres et formulaires améliorés.
Planning et Pointage	Ajout des plans permanents, récurrences, vues calendrier, rapports et calculs de présence liés au shift.

Phase	Travaux réalisés ou prévus
Support et tâches	Transformation du contact RH en tickets et enrichissement du module Task Keep.
Sécurité et IA	Brevo, reset sécurisé, approbation de compte, Gemini, RAG et filtrage par rôle.
Tests et validation	Tests ciblés sur Planning, comptes, Brevo, assistant, permissions et sécurité.
Rédaction	Mise à jour du rapport initial en conservant les modules encore valides et en retirant seulement les éléments supprimés.

1.7 Conclusion

Ce chapitre a présenté le cadre général du projet et la nécessité d'une plateforme RH centralisée. L'étude de l'existant montre que la solution doit être à la fois fonctionnelle, sécurisée, ergonomique et évolutive. Le chapitre suivant détaille les besoins actualisés de la plateforme.

Chapitre 2

Spécification des besoins

2.1 Introduction

Dans ce chapitre, nous décrivons les besoins fonctionnels et non fonctionnels de la plateforme. Les besoins sont déduits des modèles, formulaires, vues, routes, services et tests du dépôt, ainsi que du résumé de développement fourni pour la dernière phase.

2.2 Besoins fonctionnels

La plateforme doit permettre l'authentification des utilisateurs, la gestion des profils et la redirection vers un tableau de bord adapté. Le module `accounts` définit quatre rôles : administrateur, responsable RH, responsable hiérarchique et employé. Le profil relie un utilisateur Django à un employé et indique si le compte est actif.

Le système doit permettre la création de compte sans intervention manuelle directe. Un visiteur soumet une demande, reçoit un code de vérification par Brevo, valide son adresse, puis attend la décision de l'administrateur. Après approbation, le compte est créé et l'onboarding RH peut démarrer. Le mot de passe oublié suit le même principe de vérification par code.

La plateforme doit gérer les employés, les départements, les services et les postes. Les améliorations récentes concernent la recherche par nom, les filtres, l'édition sans réinitialisation des dates, les formulaires plus propres, la création dédiée des départements, services et postes, ainsi que les suggestions de niveaux.

Le système doit gérer les congés avec un workflow enrichi. La demande peut nécessiter une validation du manager puis une validation RH. L'employé peut suivre les états d'attente, d'acceptation ou de refus. Les managers ne doivent voir que les demandes des employés qu'ils supervisent.

Les demandes administratives doivent fonctionner comme un échange structuré.

Le système propose une page de détail, des réponses, des pièces jointes, des images, des filtres et une séparation entre demandes en attente, approuvées et rejetées.

Le Pointage doit enregistrer l'entrée et la sortie, puis calculer les heures à partir du shift planifié. Il doit gérer les heures manquantes, les retards, les sorties anticipées, les heures supplémentaires et les absences. En l'absence de shift planifié, le système doit afficher une information claire sans produire un calcul erroné.

Le Planning doit devenir un module complet : vue d'ensemble, calendrier, vue journalière, hebdomadaire, bihebdomadaire, mensuelle, gestion des shifts, planning permanent, récurrence, rapports, paramètres, filtres, cartes KPI, création groupée, copie, conflits et API JSON.

Le module Task Keep doit permettre au manager de déléguer des tâches directes, multiples, ouvertes ou automatiques. Les tâches doivent posséder priorité, échéance, conversation, validation, feedback, archive, points et éventuellement lien avec un shift.

Le support RH doit remplacer les anciennes réclamations isolées. Il doit fonctionner comme un système de tickets avec historique, pièces jointes, images, participants, affectation RH, statut, clôture, notation, récompenses et classement mensuel.

L'assistant intelligent doit aider les utilisateurs dans l'application. Il doit utiliser Gemini avec une approche RAG sécurisée, tenir compte du rôle, guider la navigation, répondre sur Planning, tâches, congés, support et pointage, refuser les demandes de modification directe, protéger la base de données et filtrer les informations sensibles.

L'administration doit devenir un module mère avec plusieurs onglets : gestion des utilisateurs, rôles, permissions, audit, sécurité, demandes de base, demandes de compte, rapports et paramètres. Elle doit permettre l'approbation des comptes, l'amélioration de l'audit, la configuration Brevo et le suivi des actions sensibles.

2.3 Besoins non fonctionnels

- Sécurité : les vues sensibles doivent être protégées par authentification, rôle et contrôle métier.
- Confidentialité : un utilisateur ne doit consulter que les données autorisées par son rôle et son périmètre.
- Traçabilité : les opérations importantes doivent alimenter l'audit.
- Fiabilité : les traitements critiques doivent utiliser validations backend et gestion d'exceptions.
- Ergonomie : les interfaces doivent être compactes, professionnelles, cohérentes et peu encombrées.
- Maintenabilité : les traitements doivent rester séparés entre vues, formulaires,

modèles et services.

- Testabilité : les scénarios sensibles doivent être couverts par des tests Django ciblés.
- Résilience : l'assistant doit fournir une réponse de secours si Gemini n'est pas disponible.

2.4 Présentation des acteurs

TABLE 2.1 – Acteurs du système actualisé

Acteur	Rôle dans le système
Administrateur	Gère les utilisateurs, rôles, permissions, demandes de compte, paramètres, audit, sécurité et rapports.
Responsable RH	Suit les employés, congés, demandes administratives, formations, tickets RH, support et données opérationnelles.
Responsable hiérarchique	Suit son équipe, valide les congés de son périmètre, délègue les tâches et consulte planning et pointages.
Employé	Consulte son profil, son planning, son pointage, ses demandes, ses tâches, ses formations, ses tickets et ses points.
Assistant Gemini	Répond aux questions autorisées, guide la navigation et refuse les demandes dangereuses ou hors périmètre.
Brevo	Service externe utilisé pour l'envoi des codes de vérification, création de compte et mot de passe oublié.

2.5 Description des principaux cas d'utilisation

TABLE 2.2 – Cas d'utilisation actualisés

Cas	Description
Créer un compte	Le visiteur soumet une demande, vérifie son adresse par Brevo, puis l'administrateur approuve ou refuse.
Réinitialiser le mot de passe	L'utilisateur reçoit un code de vérification, le confirme, puis définit un nouveau mot de passe.
Gérer employés et organisation	RH ou admin crée et modifie employés, départements, services, postes, niveaux et responsables.

Cas	Description
Traiter un congé	L'employé soumet une demande, le manager valide selon le périmètre, puis le RH finalise.
Traiter une demande administrative	Le responsable consulte le détail, répond, joint des fichiers et change le statut.
Planifier les shifts	RH ou admin crée shifts normaux, plans permanents, récurrences, pauses et vérifie les conflits.
Pointer entrée et sortie	L'employé pointe ; le service calcule les indicateurs en fonction du shift planifié.
Gérer une tâche d'équipe	Le manager affecte, suit, valide, commente, archive et attribue éventuellement des points.
Ouvrir un ticket RH	L'employé ouvre un ticket, ajoute pièces jointes, échange avec RH, puis le ticket est clôturé et noté.
Utiliser l'assistant	L'utilisateur pose une question ; le backend filtre le contexte autorisé et répond avec Gemini ou fallback.

2.6 Diagramme global des cas d'utilisation

Acteur	Cas d'utilisation principaux
Employé	S'authentifier, consulter dashboard, pointer, consulter planning, demander congé, demander compte, ouvrir ticket, suivre tâches, utiliser assistant.
Manager	Consulter équipe, valider congés, créer tâches, suivre pointages, consulter planning, utiliser assistant.
Responsable RH	Gérer employés, traiter demandes, gérer tickets, formations, planning, pointage et rapports.
Administrateur	Gérer comptes, rôles, permissions, audit, sécurité, paramètres Brevo, rapports et approbations.
Services externes	Brevo envoie les courriels transactionnels ; Gemini produit les réponses IA filtrées.

FIGURE 2.1 – Diagramme global des cas d'utilisation actualisé

2.7 Conclusion

Ce chapitre a détaillé les besoins fonctionnels, non fonctionnels, les acteurs et les principaux cas d'utilisation. Les exigences montrent que la plateforme couvre désormais un périmètre RH plus professionnel : onboarding, planning, pointage, tâches, ti-

ckets, administration, sécurité et assistant intelligent.

Chapitre 3

Conception du système

3.1 Introduction

Ce chapitre présente la conception de la plateforme RH Django. Il décrit l'organisation fonctionnelle du système, la structure des données, l'architecture logicielle et les choix techniques retenus pour relier les différents modules : employés, congés, pointage, planning, tâches, tickets RH, boutique, notifications, audit, administration, Brevo et assistant Gemini.

3.2 Vue fonctionnelle

3.2.1 Organisation des modules

La conception fonctionnelle adopte une logique de navigation par onglets mères et onglets fils. Cette organisation évite la multiplication de liens dans la sidebar et permet de regrouper les fonctionnalités par domaine métier. Par exemple, Planning devient un module mère avec plusieurs vues enfants, Administration regroupe les onglets de sécurité et de comptes, et Organisation regroupe employés, départements, services et postes.

Module mère	Onglets ou fonctions enfants
Organisation RH	Employés, départements, services, postes, niveaux, hiérarchie.
Demandes	Congés, demandes administratives, validation manager, validation RH.
Planning	Overview, calendrier, jour, semaine, deux semaines, mois, shifts, rapports, paramètres.
Présence	Pointage, historique, comparaison prévu/réel, retards, heures manquantes.
Tâches	Tâches directes, ouvertes, multiples, validation, archives, points, feedback.
Support RH	Tickets, conversations, pièces jointes, participants, affectation, clôture, notation.
Administration	Utilisateurs, rôles, permissions, demandes de compte, audit, sécurité, rapports, paramètres.
Assistant	Gemini, RAG, guide applicatif, assistant Planning, filtrage par rôle.

FIGURE 3.1 – Vue fonctionnelle par modules mères et onglets fils

3.2.2 Éléments retirés de la conception

La mise à jour retire uniquement les modules ou intégrations qui ne correspondent plus à l'application actuelle. Le module Documents autonome n'est plus présenté comme un espace indépendant, car les pièces jointes sont gérées dans les tickets, conversations et messages. Le module Réclamation RH autonome est remplacé par le support RH professionnel. La vérification Outlook est remplacée par Brevo. L'ancienne API IA est remplacée par Gemini.

3.3 Vue statique

3.3.1 Diagramme de classes métier principal

Le diagramme de classes principal se concentre sur les entités réellement utilisées dans la version actuelle. Employé reste le pivot du domaine RH. Il est relié à l'organisation, aux congés, aux pointages, aux shifts planifiés, aux tâches, aux tickets, aux formations, aux points et aux actions d'audit.

TABLE 3.1 – Dictionnaire de données synthétique actualisé

Entité	Attributs clés	Rôle
UtilisateurProfile	role, actif, employe	Lie un compte Django à un rôle métier.
AccountCreation Request	email, status, employee, user	Gère la demande de création de compte.
VerificationCode	purpose, code hash, expires at	Sécurise la création de compte et le reset.
Employe	matricule, nom, email, responsable	Porte les données RH et la hiérarchie.
Departement, Service, Poste	nom, niveau, relations	Structurent l'organisation interne.
DemandeConge	type, dates, statut, validation	Gère le cycle des congés.
Demande Administrative	type, statut, réponses	Gère les demandes internes.
PlanningShift	type, dates, récurrence, pause	Planifie les horaires et les plans permanents.
Pointage	entrée, sortie, shift, heures	Compare le réel au planning prévu.
TacheEquipe	mode, priorité, statut, manager	Gère la délégation et le suivi des tâches.
ConversationRH	numéro ticket, catégorie, statut	Représente le support RH par ticket.
MessageRH	contenu, pièce jointe, auteur	Stocke les échanges et fichiers du ticket.
HistoriqueAction	action, détails, utilisateur	Assure la traçabilité.
ComptePoints	solde points	Stocke les points de l'employé.
TransactionPoints	source, points, solde	Trace gains, achats et remboursements.

3.3.2 Classes assistant, tickets et comptes

Les classes liées à l'assistant ne décrivent plus un simple chatbot expérimental. Le composant courant récupère un contexte autorisé selon le rôle de l'utilisateur, refuse les demandes de mutation et délègue la génération à Gemini lorsque la clé est configurée. Les comptes utilisent une demande vérifiée par courriel puis approuvée administrativement. Les tickets remplacent les réclamations et rassemblent conversations, pièces jointes, participants et notation.

Domaine	Classes ou services liés
Comptes	AccountCreationRequest, VerificationCode, AccountRequestService, BrevoEmailService.
Assistant	core.ai_assistant, retrievers, contexte autorisé, fallback local, appel Gemini.
Tickets RH	ConversationRH, MessageRH, SupportRHRe-ward, participants, responsable RH.
Planning	PlanningShift, planning services, assistant Planning, API JSON.
Pointage	Pointage, pointer_entree, pointer_sortie, liaison shift planifié.

FIGURE 3.2 – Classes et services ajoutés ou renforcés

3.4 Vue applicative

3.4.1 Architecture logicielle

L'architecture logicielle repose sur une application Django structurée en couches. Les templates affichent les données et déclenchent les actions. Les vues vérifient la session, récupèrent le profil, appliquent les permissions et appellent les services. Les services portent les traitements métier : planning, pointage, tickets, tâches, transactions de points, courriels Brevo, audit et assistant. L'ORM persiste les données dans la base.

Couche	Responsabilité
Interface	Templates Django, Bootstrap, JavaScript, filtres, modals, offcanvas, sidebar.
Vues	Contrôle de session, permissions, orchestration des workflows.
Services	Règles métier, transactions, courriels, pointage, planning, assistant, audit.
Modèles	Entités RH, relations, contraintes et validations.
Services externes	Brevo pour les courriels transactionnels et Gemini pour l'assistant.

FIGURE 3.3 – Architecture applicative actualisée

3.5 Vue dynamique

3.5.1 Séquence création de compte

1	Le visiteur remplit le formulaire de demande de compte.
2	Le backend valide le domaine, la force du mot de passe et l'unicité des informations.
3	Brevo envoie un code de vérification à l'adresse fournie.
4	Le visiteur saisit le code ; la demande devient vérifiée.
5	L'administrateur approuve ou refuse dans le module Administration.
6	En cas d'approbation, le compte utilisateur et l'onboarding RH sont créés.

FIGURE 3.4 – Séquence création de compte avec Brevo

3.5.2 Séquence Planning vers Pointage

1	RH ou admin crée un shift normal ou un plan permanent.
2	Le service vérifie les conflits, congés validés, pauses et règles de récurrence.
3	L'employé pointe son entrée ; le système recherche le shift applicable.
4	À la sortie, le service compare horaires réels et horaires planifiés.
5	Le système calcule retard, sortie anticipée, heures manquantes, heures supplémentaires et points.

FIGURE 3.5 – Séquence du lien Planning et Pointage

3.5.3 Séquence assistant Gemini

1	L'utilisateur pose une question depuis l'interface assistant.
2	Le backend identifie le rôle et refuse les demandes de mutation ou les requêtes sensibles.
3	Les retrievers récupèrent uniquement le contexte autorisé.
4	Gemini reçoit un prompt borné ; si indisponible, une réponse locale de secours est générée.
5	La réponse indique les informations utiles et, si possible, les actions de navigation.

FIGURE 3.6 – Séquence de l'assistant Gemini sécurisé

3.6 Choix technologiques

TABLE 3.2 – Technologies utilisées dans le projet

Technologie	Usage	Justification
Django	Framework web	Architecture robuste, ORM, sécurité intégrée, rapidité de développement.
SQLite	Base locale	Simple pour le développement et les tests.
Bootstrap	Interface	Composants responsives et cohérents.
JavaScript	Interactivité	Planning, modals, filtres, assistant et actions dynamiques.
Brevo	Courriels	Codes de vérification et courriels transactionnels.
Gemini	Assistant IA	Réponses contextualisées avec meilleure évolutivité.
RAG	Recherche de contexte	Filtrage des informations autorisées avant réponse IA.
Tests Django	Validation	Vérification des workflows sensibles et des permissions.

3.7 Conclusion

Ce chapitre a mis à jour la conception du système. Les diagrammes et tableaux ne présentent plus les modules supprimés comme des fonctionnalités actuelles. Ils mettent en avant les blocs réellement présents : comptes Brevo, Planning avancé, Pointage lié, Task Keep, Support RH, Administration et assistant Gemini.

Chapitre 4

Réalisation du système

4.1 Introduction

Ce chapitre présente l'environnement de développement, les technologies utilisées, la structure du code, les modules implémentés, les tests, les interfaces, le déploiement et les difficultés rencontrées. Il conserve la logique du rapport initial mais remplace les descriptions obsolètes par l'état courant de l'application.

4.2 Environnement de développement

4.2.1 Environnement matériel

Le projet a été développé sur un poste de travail personnel permettant l'exécution de Django, de SQLite, des tests automatisés et du serveur local. Cet environnement suffit pour la validation fonctionnelle, l'analyse du code, les démonstrations locales et la génération du rapport.

4.2.2 Environnement logiciel

L'environnement logiciel repose sur Python, Django, SQLite, les templates Django, Bootstrap, JavaScript, les outils de tests intégrés à Django, Brevo pour les courriels transactionnels et Gemini pour l'assistant intelligent. La configuration utilise des variables d'environnement pour les clés sensibles.

4.3 Architecture du projet

La structure du code met en évidence une séparation des responsabilités. Les modèles de données sont dans `hr/models.py` et `accounts/models.py`. Les formulaires et validations sont dans les formulaires Django. Les règles d'autorisation sont dans les permissions et les profils. Les traitements métier atomiques sont dans les services. Les routes sont déclarées dans `hr/urls.py`, `accounts/urls.py` et `core/urls.py`. Les vues assurent le lien entre requêtes, formulaires, services et templates.

4.4 Implémentation des modules principaux

4.4.1 Authentification, comptes et Brevo

L'authentification repose sur le système intégré de Django. La mise à jour importante concerne la création de compte et le mot de passe oublié. La création de compte passe par une demande, une vérification par code envoyé avec Brevo, puis une approbation administrateur. Le mot de passe oublié suit un workflow similaire afin d'éviter la modification directe sans contrôle. Les paramètres Brevo, les durées de validité, les tentatives et les délais de renvoi sont configurables.

4.4.2 Gestion des employés et de l'organisation

Le module des employés gère les informations personnelles, l'affectation à un département, un service, un poste et un responsable. Les améliorations portent sur la recherche par nom, les filtres, l'édition fiable des dates, les formulaires plus propres et les validations. La création des départements, services et postes est isolée dans un onglet dédié. Les niveaux bénéficient de suggestions et d'une saisie libre.

4.4.3 Congés et demandes administratives

Le module Congés gère désormais une logique de validation plus complète : validation manager puis validation RH lorsque le workflow l'exige. Le manager ne consulte que les employés de son périmètre. Les demandes administratives disposent d'une page de détail, de réponses, de pièces jointes, d'images, d'un tri par statut et d'un comportement proche d'un échange professionnel.

4.4.4 Planning avancé

Le module Planning a été fortement enrichi. Il propose une vue d'ensemble, un calendrier, des vues journalière, hebdomadaire, bihebdomadaire et mensuelle, des statistiques, des filtres, des cartes KPI, des rapports et des paramètres. Les shifts peuvent être normaux ou permanents. Les plans permanents supportent des règles de récurrence, des pauses et une heure de fin standard. Les services vérifient les conflits, les chevauchements et les congés validés.

4.4.5 Pointage intégré au Planning

Le Pointage ne calcule plus seulement une durée brute. Il recherche le shift applicable au moment de l'entrée ou de la sortie et compare les horaires réels avec les horaires prévus. Il peut indiquer retard, sortie anticipée, heures manquantes, heures supplémentaires et absence. Cette liaison réduit les erreurs de calcul et rend les données de présence plus exploitables.

4.4.6 Task Keep

Le module Task Keep permet au manager de créer des tâches directes, multiples, ouvertes ou automatiques. Les tâches possèdent priorité, échéance, état, conversation, validation, feedback, archive et points. Cette logique permet de relier la hiérarchie Direction, Manager et Employé à des objectifs opérationnels concrets.

4.4.7 Support RH par tickets

L'ancien contact RH et le module de réclamation isolé sont remplacés par un support RH professionnel. Les tickets disposent d'un numéro, d'un sujet, d'une catégorie, d'une priorité, d'un statut, d'un responsable RH, de participants et d'un historique. Les messages peuvent contenir des pièces jointes et des images. Les tickets peuvent être clôturés, notés et utilisés pour calculer des indicateurs RH.

4.4.8 Administration

L'administration devient un module mère. Elle regroupe les utilisateurs, rôles, permissions, demandes de compte, audit, sécurité, rapports et paramètres. L'administrateur peut approuver les demandes de compte, ajuster les paramètres de vérification, contrôler les accès et consulter l'historique des actions.

4.4.9 Assistant Gemini

L'assistant global est exposé par le backend et fonctionne avec Gemini lorsque la configuration est disponible. Avant l'appel IA, le système classe l'intention, refuse les mutations dangereuses, bloque les tentatives d'injection, vérifie les droits sur les zones sensibles, récupère un contexte autorisé et prépare un prompt borné. En cas d'indisponibilité de l'API, un fallback local répond de manière prudente.

4.4.10 Dashboard, navigation et UI/UX

L'interface a été améliorée autour d'une approche entreprise : cartes KPI compactes, filtres plus clairs, formulaires plus propres, couleurs plus cohérentes, sidebar qui conserve son état, onglets mères et fils, réduction du scrolling et mise en valeur des modules les plus utilisés.

4.5 Tests et validation

TABLE 4.1 – Tests et validations réalisés

Module	Scénario	Résultat
Django	<code>python manage.py check</code>	Aucun problème détecté.
Planning	Tests ciblés PlanningModuleUpgradeTests	15 tests réussis.
Assistant	Tests core de sécurité et permissions	12 tests réussis.
Comptes et Brevo	Tests comptes, codes, reset et approbation	7 tests réussis.
Pointage	Liaison avec shift planifié	Conforme dans les tests Planning.
Sécurité	Refus des mutations et filtrage des rôles	Conforme dans les tests assistant.

4.6 Principales interfaces graphiques

4.6.1 Tableau de bord

Le tableau de bord affiche les indicateurs clés selon le rôle connecté : employés actifs, demandes, pointages, tâches, tickets, formations, points et actions rapides. Les

cartes statistiques sont compactes et cohérentes.

4.6.2 Organisation RH

Les interfaces d'employés, départements, services et postes permettent la recherche, le filtrage, la création et l'édition. L'amélioration la plus importante concerne la fiabilité de l'édition et la séparation claire des créations d'entités organisationnelles.

4.6.3 Présence / Pointage

L'interface Pointage montre le shift du jour, les actions d'entrée et sortie, l'historique, les heures prévues et les heures réalisées. Elle prépare l'exploitation des données pour les rapports et le contrôle RH.

4.6.4 Planning

L'interface Planning propose plusieurs vues : overview, calendrier, jour, semaine, deux semaines, mois, shifts, attendance, leave, tasks, approvals, reports et settings. Les actions de création, modification, déplacement et consultation sont regroupées autour de composants dynamiques.

4.6.5 Tâches et Support

Les interfaces de tâches et de tickets mettent en avant les workflows opérationnels : affectation, acceptation, conversation, pièces jointes, validation, clôture, notation et archives.

4.7 Déploiement

Pour un déploiement réel, il faut désactiver `DEBUG`, externaliser `SECRET_KEY`, configurer `ALLOWED_HOSTS`, utiliser une base serveur comme PostgreSQL, collecter les fichiers statiques, gérer les médias, configurer Brevo, fournir une clé Gemini si l'assistant doit appeler l'API et protéger les paramètres sensibles.

4.8 Difficultés rencontrées et solutions apportées

La première difficulté concerne la taille de l'application. Elle intègre employés, congés, demandes, pointage, planning, tâches, tickets, points, boutique, formations, administration et assistant. La solution retenue consiste à maintenir une séparation claire entre applications Django, services métier, formulaires, permissions et templates.

La deuxième difficulté concerne les rôles et permissions. Chaque acteur doit accéder uniquement aux informations correspondant à son profil. Le système utilise donc les profils, les fonctions de permission, les décorateurs et les filtres de périmètre.

La troisième difficulté concerne l'alignement entre le rapport et le code réel. Le PDF initial contenait des descriptions devenues obsolètes. La mise à jour retire uniquement les éléments supprimés et ajoute les fonctionnalités réellement développées.

4.9 Conclusion

Ce chapitre a présenté la réalisation actuelle de la plateforme. Les modules conservés du rapport initial restent présents, tandis que les modules supprimés sont remplacés par les nouveaux workflows : Brevo, Gemini, Planning avancé, Pointage lié, Task Keep, Support RH par tickets et Administration.

Conclusion générale

Ce projet a permis de concevoir et de faire évoluer une plateforme web de gestion des ressources humaines avec Django. L'application centralise les opérations RH essentielles : authentification, rôles, employés, organisation, congés, demandes administratives, pointage, planning, tâches, formations, messages RH, support par tickets, actualités, boutique, points, analyses salariales, notifications, audit, administration et assistant intelligent.

La mise à jour du rapport respecte le contenu initial tout en corrigeant les parties qui ne correspondent plus à l'application actuelle. Les modules encore présents sont conservés. Le module Documents autonome est retiré au profit des pièces jointes contextuelles. Le module Réclamation RH autonome est remplacé par les tickets RH. L'ancienne vérification Microsoft Outlook est remplacée par Brevo. L'ancienne intégration IA est remplacée par Gemini avec RAG sécurisé.

Les résultats obtenus montrent que la plateforme ne se limite pas à des pages CRUD. Elle intègre des règles métier importantes : contrôle des profils actifs, limitation des accès par rôle, validation des dates, workflows manager et RH, transaction de points, contrôle du stock, gestion des tickets, liaison Planning et Pointage, notifications, journalisation et sécurité de l'assistant.

Les perspectives d'amélioration concernent l'ajout d'exports avancés pour les rapports, l'amélioration des tableaux analytiques, l'optimisation du déploiement en production, l'extension mobile et l'enrichissement progressif des assistants spécialisés.

Annexe A

Annexes

A.1 Routes principales

TABLE A.1 – Extrait des routes applicatives actualisées

Route	Fonction
/login	Authentification utilisateur.
/creer-compte	Demande de création de compte.
/creer-compte/verification	Vérification du code Brevo.
/mot-de-passe-oublie	Démarrage du reset mot de passe.
/dashboard	Tableau de bord adapté au rôle.
/admin	Administration, comptes, audit, sécurité et paramètres.
/employes	Liste, recherche et édition des employés.
/conges	Suivi et traitement des congés.
/demandes	Demandes administratives.
/pointage	Entrée, sortie et suivi de présence.
/planning	Planning avancé et shifts.
/taches	Tâches d'équipe.
/messages-rh	Support RH et tickets.
/assistant/chat	Assistant Gemini sécurisé.
/formations/mes-formations	Formations affectées à l'employé.
/actualites	Actualités internes.
/boutique	Boutique et commandes matériel.
/audit	Historique d'audit.

A.2 Éléments supprimés ou remplacés

TABLE A.2 – Éléments retirés de l'ancien rapport

Ancien élément	Statut	Remplacement
Documents auto-nome	Supprimé	Pièces jointes dans tickets, conversations et messages.
Réclamation RH autonome	Supprimé	Support RH professionnel sous forme de tickets.
Microsoft Outlook vérification	Supprimé	Brevo Transactional Emails.
Ancienne API IA	Supprimée	Gemini API avec RAG sécurisé et fallback local.

A.3 Commandes utiles

```
python manage.py check
python manage.py test hr.tests.PlanningModuleUpgradeTests --verbosity 2
python manage.py test core.tests --verbosity 2
python manage.py test accounts.tests --verbosity 2
python manage.py runserver
python manage.py seed_demo_rh
```